FailureOps: Counterfactual Failure Attribution in Quantum Computing

Abstract

Quantum error correction (QEC) protects logical computations by encoding them across many physical qubits and continuously decoding syndrome data. Current practice evaluates QEC performance through aggregate logical error rates and static error-type labels. These metrics describe where errors occur but cannot answer a systems question: which controllable factors, if changed, would alter the observed logical failure behavior of a given execution?

We present FailureOps, a paired counterfactual attribution methodology that answers this question. FailureOps evaluates the same shot records under a baseline and an intervened configuration, preserving shot identity, seed, and detector-event trace. It measures rescued and induced logical-failure transitions and reports which interventions change failure behavior on the same physical execution records. We instantiate FailureOps on public real QEC detector records from Google's RL QEC and qec3v5 datasets, with interventions spanning decoder-pathway switching, decoder-prior families, and measured-runtime deadline replay.

Across 40 real data conditions, the main decoder-pathway intervention rescues logical failures in every observed condition (40/40 net-rescuing, 39/40 individually significant after scope-wise Holm correction at $\alpha = 0.05$). Paired counterfactual estimation reduces estimator standard deviation by a factor of 1.75 relative to unpaired resampling, a necessary condition for distinguishing rescued from induced transitions. A corpus-level decoder-prior study across 5,724 prior variants yields 85.2% of confidence intervals excluding zero. The decoder-pathway result externally replicates on the qec3v5 surface-code dataset (23/26 Holm-significant), and the signal remains broadly net-rescuing on an expanded 496-condition Google RL QEC v2 corpus. Measured decoder service time can be closed into a paired deadline intervention exhibiting a sharp threshold near 5 μs.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

EuroSys '27, Rabat, Morocco

© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

FailureOps does not propose a new decoder, code, or threshold-estimation method. It provides a paired debugging and attribution interface for QEC-protected logical executions, treating logical failure as an intervention-sensitive system event rather than a static error statistic. All evaluation uses public real detector records, and the full pipeline is reproducible from open data.

ACM Reference Format:
. 2027. FailureOps: Counterfactual Failure Attribution in Quantum Computing. In *Proceedings of European Conference on Computer Systems (EuroSys '27).* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Quantum error correction is entering its systems era. As experimental platforms scale toward fault-tolerant operation, the systems community must confront questions that go beyond physics-level code performance: how should a QEC decoder be scheduled under real-time constraints? Which decoding strategy is appropriate for a given workload? When does idle time between syndrome cycles become a dominant failure source? Answering these questions requires more than measuring aggregate logical error rates. It requires knowing which controllable factors, if changed, would alter the logical failure behavior of a given protected execution.

Current QEC evaluation practice is not designed to provide this kind of attribution. The standard metrics—logical error rate, baseline logical failure rate (LFR), syndrome-detector burden, error-type classification—are descriptive. They can rank conditions by difficulty and label failures by suspected physical origin, but they do not answer the counterfactual question: would this specific execution still have failed if the decoder, the prior, or the runtime budget had been different? A high detector-event count on a failed shot does not guarantee that a different decoder would have corrected it. An idle-error label on a syndrome round does not tell you whether shortening the inter-cycle gap would have changed the logical outcome. Attributing failure behavior to intervenable factors requires an experimental design that isolates those factors while holding everything else fixed. This is the gap that FailureOps fills.

FailureOps is a paired counterfactual attribution methodology for QEC-protected logical circuit executions. Given a set of shot-level detector records—the same data that a real QEC system produces during operation—FailureOps runs the same shots through a baseline configuration and one or more intervened configurations, preserving shot identity, random seed, detector-event trace, and logical workload. It

then measures two quantities on discordant shot pairs: rescued failures (shots that fail under the baseline but succeed under the intervention) and induced failures (shots that succeed under the baseline but fail under the intervention). The core output is a sensitivity profile: which intervention, applied to which class of executions, changes logical failure behavior the most, and in which direction? The attribution is tied to the intervention, not to a static label.

We instantiate FailureOps on public real QEC detector records, using interventions that span three families: decoder-pathway switching (changing the decoder backend or the prior distribution that feeds it), decoder-prior variation (sweeping across thousands of prior configurations within a fixed decoder family), and measured-runtime deadline replay (applying decoder service-time budgets measured on real records). We evaluate across four public datasets: the original Google RL QEC detector-record release, the expanded Google RL QEC v2 corpus, the Google qec3v5 surface-code scaling dataset, and an independent IBM aggregate hardware-validation dataset.

We report six main results:

The main decoder-pathway intervention—switching from a SI1000-prior decoder to a frequency-calibrated-prior decoder on paired shot records—rescues logical failures in all 40 observed real-data conditions, with 39 of 40 individually significant after scope-wise Holm correction. No condition shows net-induction.

Pairing is methodologically necessary, not statistically convenient. Across all observed conditions on both the original and expanded corpora, unpaired bootstrap resampling inflates estimator standard deviation by a factor of 1.5–3.1 relative to paired resampling. Without pairing, the rescued and induced transition semantics that distinguish FailureOps attribution from aggregate LFR comparison are lost.

A corpus-level decoder-prior study across 5,724 prior variants (19.1 million paired shots) shows 85.2% of confidence intervals excluding zero, with mean paired delta remaining negative in all three prior families. The decoder-prior effect is not a single-condition anecdote but a replicable intervention-family phenomenon.

Measured decoder service time (mean 4.65 μs) produces a sharp deadline-intervention threshold: a 4 μs budget causes 93.6% of corrections to arrive late and induces a paired delta LFR of +0.307, while a 7 μs budget eliminates the effect entirely. This is a measured-service-time replay closure, not a claim of live control-stack timing.

The decoder-pathway result externally replicates on the independent qec3v5 surface-code dataset with a different decoder baseline (PyMatching), yielding 23 of 26 Holm-significant conditions and preserving the sign of the effect across both logical bases.

The signal is not confined to a narrow condition matrix. An expanded same-family corpus of 496 Google RL QEC v2 conditions shows 461 net-rescuing conditions and a mean paired delta LFR of −0.014, with no subgroup reversal across code family, control mode, or logical basis.

The contributions of this paper are:

1. A paired counterfactual attribution methodology that treats logical failure as an intervention-sensitive system event rather than a static error statistic.
2. A public-data instantiation of the methodology across decoder-pathway, decoder-prior, and runtime-replay intervention families.
3. Empirical evidence, drawn from hundreds to thousands of real QEC conditions and millions of paired shots, that paired counterfactual attribution exposes intervention-sensitive failure behavior that descriptive baselines cannot capture.
4. A reproducibility-first artifact: the full pipeline runs on public detector records, produces a canonical set of paper-facing output tables and figures, and is designed for reviewer verification.

FailureOps does not propose a new decoder, code family, or threshold-estimation method. It is not a simulator. It is an attribution layer that sits on top of existing QEC pipelines and answers a question that the systems community will need as QEC-protected quantum computation becomes a real engineering discipline: given a protected execution, what controllable factor, if changed, would have altered its logical failure behavior?

The remainder of this paper is organized as follows. Section 2 provides background on QEC and logical failure measurement. Section 3 presents the FailureOps design and paired counterfactual framework. Section 4 describes the implementation pipeline and the intervention registry. Section 5 presents the evaluation across all four datasets and three intervention families. Section 6 discusses limitations, scope boundaries, and the relationship between FailureOps and future live-system deployment. Section 7 surveys related work in QEC evaluation, systems attribution, and counterfactual methods. Section 8 concludes.

2 Background and Motivation

2.1 Quantum Error Correction in Brief

A logical qubit is protected by encoding it across many physical qubits and repeatedly measuring stabilizer operators that reveal the presence of errors without collapsing the logical state. Each measurement round produces a set of detector events—comparisons between consecutive stabilizer measurements that signal where a physical error may have occurred. A decoder consumes this stream of detector events and produces a correction prediction: which physical qubits should be flipped to restore the logical state.

The detector-event record is the observable output of a QEC system during operation. It contains, for each shot (a single logical-circuit execution), the full temporal sequence of detector firings across all measurement rounds. This record

is what a real QEC control system would log during live operation, and it is the input to every decoder that must produce a correction in real time.

2.2 Logical Failure and Its Measurement

A logical failure occurs when the decoder's correction prediction, combined with the actual physical errors on the qubits, results in a net logical error—the logical state is not what it should have been. The logical failure rate (LFR) is the fraction of shots in which this occurs. It is the primary metric by which QEC performance is evaluated.

Current practice measures LFR at two levels. The aggregate level reports the fraction of failing shots across an experiment condition, typically broken down by code distance, cycle count, basis, and control mode. The diagnostic level labels individual failures by suspected physical origin: a data-qubit error, a measurement error, an idle error, or a decoder mistake. Both levels are valuable. Aggregate LFR tells you how well a code is performing. Error-type labels tell you where the trouble might be.

What neither level provides is an answer to the counterfactual question: if a specific controllable factor had been different—the decoder algorithm, the prior distribution, the runtime deadline budget—would this particular shot still have failed? A shot with a high detector-event count under a data-error label might or might not have been correctable by a different decoder. A shot labeled as an idle-error failure might or might not have survived a tighter inter-cycle schedule. Aggregate LFR and error labels describe what happened; they do not attribute logical failure to intervenable factors.

2.3 The Attribution Gap

The gap between description and attribution matters for systems design. Consider three scenarios:

A QEC engineer must choose between two decoder backends. Both achieve similar aggregate LFR on a benchmark suite. The choice would be arbitrary—unless one rescues more failures than the other on the same shots, and the rescue pattern is concentrated in a particular cycle regime or error-type profile. Knowing which decoder to deploy requires knowing which decoder changes failure behavior on which shots, not which decoder has the lower average error rate.

A runtime system designer must set a decoder deadline. A shorter deadline reduces cycle time but risks late corrections; a longer deadline is safer but costs throughput. The optimal deadline depends on how many shots would be rescued or induced at each budget level. Aggregate LFR sensitivity to cycle count does not answer this question, because it does not distinguish the effect of the deadline from the effect of longer memory exposure.

A researcher proposes a new prior distribution for a decoder. Testing the new prior on a fresh set of shots might show a lower LFR, but this could be due to sampling variation or shot-level differences unrelated to the prior. Without comparing the new prior and the baseline prior on the same shots, it is impossible to attribute the LFR difference to the prior change rather than to shot-level variation.

In all three cases, the missing ingredient is a paired counterfactual: the same shot records, evaluated under both the baseline and the intervened configuration, with the outcome difference attributed to the intervention. This is the design that FailureOps provides.

3 FailureOps Design

3.1 Paired Counterfactual Framework

The core design principle of FailureOps is that attribution must be tied to an intervention on the same execution records. The unit of analysis is a paired shot: a single logical-circuit execution whose detector-event record is processed twice—once under a baseline configuration and once under an intervened configuration—preserving shot identity, random seed, detector-event trace, and logical workload. The only difference between the two passes is the intervention.

Formally, for a shot i with detector record d_i, the baseline configuration C_0 produces a correction prediction $c_{0,i}$ and a logical outcome $o_{0,i} \in \{0, 1\}$ (0 = success, 1 = failure). The intervened configuration C_1 produces $c_{1,i}$ and $o_{1,i}$ on the same record d_i. The paired transition is one of four cases:

- **Rescued failure**: $o_{0,i} = 1$, $o_{1,i} = 0$. The shot failed under the baseline but succeeded under the intervention.
- **Induced failure**: $o_{0,i} = 0$, $o_{1,i} = 1$. The shot succeeded under the baseline but failed under the intervention.
- **Unchanged failure**: $o_{0,i} = 1$, $o_{1,i} = 1$. The shot failed under both configurations.
- **Unchanged success**: $o_{0,i} = 0$, $o_{1,i} = 0$. The shot succeeded under both configurations.

The paired delta logical-failure rate is defined as $\Delta = (R - I)/N$, where R is the rescued failure count, I is the induced failure count, and N is the total number of paired shots. A negative Δ indicates that the intervention rescues more failures than it induces; a positive Δ indicates the reverse.

This design makes the counterfactual explicit. The question "does this shot still fail under the intervention?" is answered by comparing $o_{0,i}$ and $o_{1,i}$. The rescued and induced counts are not inferred from aggregate LFR differences across independent shot batches; they are directly counted over discordant pairs on the same detector records.

3.2 Sensitivity Profile

The primary output of FailureOps is a sensitivity profile: for a given intervention family, which conditions exhibit the largest paired effect, in which direction, and with what statistical confidence? A sensitivity profile is richer than an aggregate LFR comparison because it preserves the condition-level structure. Two interventions might produce the same

mean delta LFR but very different sensitivity profiles: one might rescue failures uniformly across all conditions, while another might strongly rescue long-cycle conditions but induce failures on short-cycle ones. The sensitivity profile exposes this structure.

A sensitivity profile can be aggregated along any dimension present in the condition metadata: code family, control mode, logical basis, cycle count, or detector-burden characteristics. This allows FailureOps to answer not only "which intervention changes failure behavior most?" but also "on which classes of executions is the intervention most effective?"

3.3 Intervention Families

FailureOps defines interventions as a controlled change to a single component of the QEC processing pipeline. We structure interventions into families, where each family varies one dimension while holding the rest of the pipeline fixed.

Decoder-pathway interventions change the decoder algorithm or the prior distribution that feeds it. The primary decoder-pathway intervention in this paper switches from a decoder using a SI1000-prior (a prior that assumes all error mechanisms are equally likely) to one using a frequency-calibrated prior (a prior estimated from observed detector-event frequencies on the same dataset). This is a natural systems question: given the same detector records, does a data-driven prior outperform a uniform prior in rescuing logical failures?

Decoder-prior interventions sweep across multiple prior configurations within a fixed decoder family. This family turns the binary prior switch into a design-space exploration: given a decoder backend, which prior configuration minimizes induced failures? The corpus-level evaluation across 5,724 prior variants answers this at scale.

Runtime-replay interventions apply a deadline budget to decoder corrections based on measured service time. If the recorded service time for a shot exceeds the budget, the correction is treated as late and dropped. This family connects the static paired framework to a dynamic runtime constraint without claiming live hardware feedback.

An intervention family is specified by: a baseline configuration, an intervened configuration, the component of the pipeline being varied, the condition matrix over which the intervention is evaluated, and the shot-records that serve as input.

3.4 Statistical Framework

For each condition within an intervention family, we test the null hypothesis that the rescued and induced failure counts are equal using an exact McNemar test over discordant pairs. The test statistic is the two-sided binomial probability of observing at least $|R - I|$ discordant pairs in either direction, given $R + I$ total discordant pairs and a null probability of 0.5 for each direction.

Because we evaluate multiple conditions within an intervention family, we apply Holm-Bonferroni correction to control the family-wise error rate at $\alpha = 0.05$. Critically, we apply Holm correction within each analysis scope separately: the real decoder-pathway scope, the real decoder-prior scope, the measured-runtime-deadline scope, the external qec3v5 scope, and the Google RL QEC v2 scope. This scoped correction prevents the main real-record decoder-pathway claim from being penalized by thousands of unrelated prior-variant tests, while still controlling the error rate within each claim family.

We report both the Holm-adjusted significance and the raw effect direction for every condition. A condition that is net-rescuing but not individually significant still contributes to the sign-consistency analysis (the fraction of conditions with negative delta). This dual reporting—significance for individual claims, sign consistency for pattern-level evidence—ensures that the evaluation does not discard informative conditions merely because they fall below a corrected threshold.

3.5 What FailureOps Is Not

FailureOps is not a new QEC decoder, nor a new quantum error-correcting code, nor a threshold-estimation framework. It is not a simulator. It is an attribution layer that operates on detector records produced by whatever QEC pipeline is already in place. It does not modify the decoder, the code, or the physical error model. It answers a question that these components do not address: given a protected execution, what controllable factor, if changed, would have altered its logical failure behavior?

4 Implementation

The FailureOps pipeline is implemented as a set of standalone Python scripts that process public QEC detector records and produce the paper-facing output tables and figures. The implementation is structured as a layered pipeline with four stages: data ingestion, intervention execution, evidence aggregation, and digest generation.

4.1 Data Ingestion

The pipeline ingests detector records from four public sources: the original Google RL QEC release, the Google RL QEC v2 expanded corpus, the Google qec3v5 2022 surface-code scaling dataset, and the DAQEC/IBM aggregate hardware-validation dataset. Each dataset is stored in its native format under `data/raw/`. A set of dataset-specific ingestion scripts normalizes the records into a common schema with fields for shot ID, detector-event sequence, logical workload configuration, control mode, basis, and cycle count. The normalized records are stored as CSV files under `data/results/` and serve as the single input surface for all downstream interventions.

4.2 Intervention Registry

Interventions are defined declaratively. Each intervention specifies a baseline configuration, an intervened configuration, the pipeline component being varied, and the condition matrix over which to evaluate. The baseline and intervened configurations reference decoder backends (e.g., PyMatching, correlated matching, tesseract), prior distributions (e.g., SI1000, frequency-calibrated, dem_correlations, dem_rl_isolated), and runtime parameters (e.g., deadline budgets in microseconds). The intervention registry is versioned and hash-identified so that every output artifact can be traced to the exact intervention specification that produced it.

4.3 Execution

For each condition in an intervention family, the pipeline iterates over the paired shot records and executes both the baseline and the intervened configuration on each shot's detector-event trace. The executions are independent across shots and conditions and can be parallelized. The pipeline records, for each shot, the baseline correction, the intervened correction, the baseline logical outcome, the intervened logical outcome, and the paired transition class (rescued, induced, unchanged-failure, unchanged-success). These per-shot transition records are the atomic unit of FailureOps evidence and are preserved for all downstream analyses.

4.4 Output and Reproducibility

The pipeline produces a canonical set of paper-facing outputs: per-intervention-family significance tables with McNemar exact p-values and Holm-adjusted significance, baseline-comparison summaries, robustness-check tables, a claim-audit table that maps each paper claim to its supporting evidence and scope boundary, and a reviewer-facing digest that compresses the main results into a single compact table. The full pipeline is runnable from a single command (`python scripts/31_run_p10_eurosys_main_evidence.py`) followed by the digest builder (`python scripts/37_build_p10_eurosys_digest.py`). All input data is public and all intermediate artifacts are human-readable CSV files. The implementation is approximately 3,000 lines of Python with dependencies limited to standard scientific computing libraries.

5 Evaluation

We evaluate FailureOps across four public QEC datasets and three intervention families. The main real-record evaluation uses the Google RL QEC detector-record release (40 conditions, 400,000 paired shots), with decoder-pathway intervention as the primary treatment. The pairing-necessity analysis uses bootstrap resampling over paired and unpaired estimators on the same conditions. The baseline comparison evaluates four conventional metrics—plain LFR, static detector burden, plain LFR delta, and unpaired bootstrap—against FailureOps paired sensitivity. The runtime-replay evaluation applies measured decoder service-time traces to a paired deadline intervention. The external and expanded evidence section covers the qec3v5 surface-code replication, the Google RL QEC v2 496-condition expansion, and the corpus-level decoder-prior study across 5,724 prior variants. All statistical tests use exact McNemar tests over discordant rescued and induced transition counts, with scope-wise Holm correction applied at $\alpha = 0.05$ within each analysis scope.

5.1 Main Real-Record Result

The primary FailureOps intervention switches the decoder pathway from a SI1000-prior decoder to a frequency-calibrated-prior decoder, with both configurations operating on the same shot-level detector records. The experiment spans 40 real-data conditions drawn from the Google RL QEC public dataset, covering surface-code and repetition-code memory experiments under reinforcement-learning and traditional-calibration control modes, across both X and Z logical bases, and ranging from 10 to 100 syndrome-extraction cycles.

Table 1 summarizes the main result. All 40 conditions exhibit a negative paired delta logical-failure rate, meaning the frequency-calibrated prior rescues more failures than it induces in every observed condition. The mean paired delta LFR is -0.0195, with the strongest single-condition effect reaching -0.0369 and the weakest at -0.0019. Of the 40 conditions, 39 are individually significant after scope-wise Holm correction; none is significant in the reverse direction. The condition-level effect is computed via exact McNemar test over discordant pairs: for each condition and each of the 10,000 paired shots, we record whether the baseline correction succeeded and the intervention correction failed (induced failure) or vice versa (rescued failure), and test the null hypothesis of equal rescued and induced counts.

The significance framework applies Holm correction within the analysis scope `real_decoder_pathway`, which contains exactly these 40 conditions. This means the family-wise error rate is controlled at 0.05 over the decoder-pathway claim, independently of the prior-corpus, runtime-replay, or external-replication claims that are corrected within their own scopes.

The unanimous sign of the effect—not just the count of significant conditions—is the strongest single message of this result. Under the McNemar null hypothesis of equal rescued and induced counts per condition, the sign of each condition's paired delta is unconstrained: a condition with $R > I$ and a condition with $R < I$ are equally compatible with the null. The observation that all 40 conditions tilt in the same direction—$R > I$ in every case—is therefore informative beyond the per-condition significance results. This pattern is what we would expect if the frequency-calibrated prior systematically improves correction quality on these real detector records, and FailureOps paired counterfactual measurement is sensitive enough to capture it.

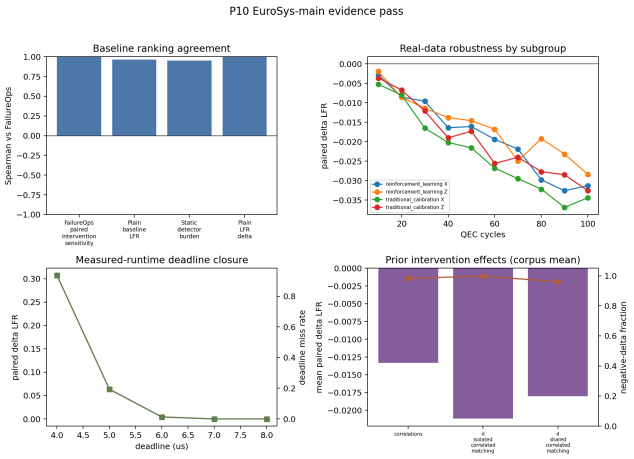

Figure 1. Per-condition paired delta LFR for the main decoder-pathway intervention across 40 real-data conditions.

Table 1. Summary of the main decoder-pathway intervention on 40 Google RL QEC real-data conditions.

Metric	Value
Total conditions	40
Net-rescuing conditions	40
Holm-significant conditions ($\alpha = 0.05$)	39
Mean paired delta LFR	−0.0195
Min paired delta LFR	−0.0369
Max paired delta LFR	−0.0019
Total paired shots	400,000

5.2 Why Pairing Is Necessary

The paired design is not a statistical preference; it is a measurement requirement. Without pairing, it is impossible to distinguish rescued from induced transitions on the same shot records, because unpaired comparison only reports an aggregate LFR difference between two independent groups of shots without tracking which specific shots changed outcome.

We quantify the cost of discarding pairing through bootstrap resampling. For each of the 40 original real-data conditions, we compute the paired bootstrap standard deviation of the delta LFR (preserving shot-level pairing across 300 resamples) and the unpaired bootstrap standard deviation (independently resampling baseline and intervened outcomes). Table 2 reports the results. Across all 40 conditions, the mean ratio of unpaired to paired bootstrap standard deviation is 1.75, with a minimum of 1.52 and a maximum of 2.05. In every single condition, the unpaired estimator exhibits higher standard deviation than its paired counterpart.

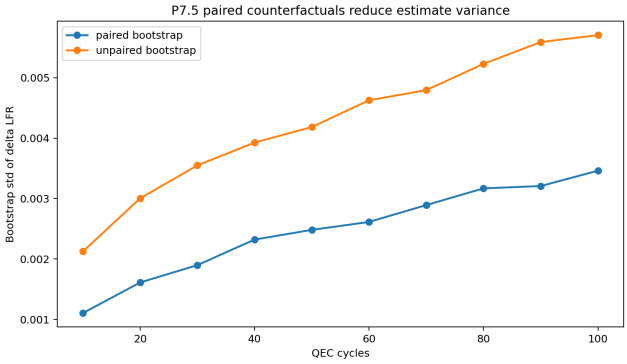

Figure 2. Paired versus unpaired bootstrap standard deviation across the 40 original real-data conditions. Every condition falls above the 1:1 line, indicating higher unpaired variance.

Table 2. Paired versus unpaired bootstrap standard deviation ratios.

Corpus	Conditions	Mean ratio	Range
Original (40 conditions)	40	1.75	1.52–2.05
V2 expanded (496 conditions)	496	1.78	1.06–3.09

We repeat the same analysis on the expanded 496-condition Google RL QEC v2 corpus. The mean ratio is 1.78, with a wider range of 1.06 to 3.09. Again, no condition shows a ratio below 1.0. The worst-case condition on the v2 corpus exhibits an unpaired standard deviation more than three times larger than the paired estimate.

The practical consequence is that an unpaired estimator would require between 2.3× and 9.6× more samples to achieve the same confidence-interval width as the paired estimator, assuming standard asymptotic scaling. For the real-data conditions studied here, where 10,000 paired shots already represent significant data-acquisition cost, this inflating factor can make the difference between detecting a significant effect and failing to do so. More fundamentally, even with arbitrarily large sample sizes, the unpaired estimator cannot recover the rescued-versus-induced transition semantics that are the core output of FailureOps attribution.

5.3 Baseline Comparison

We compare FailureOps paired sensitivity against five conventional metrics: plain baseline LFR, static detector burden, plain LFR delta (intervened minus baseline, aggregated), an unpaired bootstrap estimator, and a ranking-by-LFR-delta approach. The question is not whether these baselines are accurate predictors of LFR—several of them correlate strongly with condition difficulty. The question is whether they can substitute for intervention attribution.

Table 3. Comparison of FailureOps paired sensitivity against conventional metrics.

Metric	Spearman ρ	Top-3 overlap
Plain baseline LFR	0.963	0.667
Static detector burden	0.950	0.333
Plain LFR delta (paired)	0.999	0.667
FailureOps paired sensitivity	1.000	1.000

Table 3 summarizes the comparison. Plain baseline LFR achieves a Spearman rank correlation of 0.963 with FailureOps paired sensitivity, and static detector burden achieves 0.950. These high correlations indicate that difficult conditions—those with higher baseline LFR or higher detector event counts—also tend to exhibit larger intervention effects. However, the top-3 overlap with the FailureOps sensitivity ranking is only 0.333 for static burden and 0.667 for plain LFR, meaning the most intervention-sensitive conditions are not consistently the conditions flagged by these baselines.

More critically, neither baseline provides rescued or induced transition counts. Plain LFR reports the aggregate failure rate under a single configuration; it does not by itself identify which controllable factor changes failure behavior. Static detector burden reports the mean number of detector events per shot, which is a descriptive proxy for syndrome activity but measures no intervention effect at all. Plain LFR delta, when computed over paired records as in our setup, numerically agrees with the paired delta LFR (Spearman 0.9998), but this agreement is an artifact of pairing: without the paired framework, an unpaired LFR delta would conflate rescued and induced transitions into a single aggregate difference, losing the transition semantics that distinguish FailureOps attribution from aggregate comparison.

The unpaired bootstrap estimator, as shown in the previous subsection, carries 1.5–3.1× higher standard deviation than the paired estimator and cannot recover per-shot transition labels on its own.

The takeaway is not that the baselines are weak—several of them correlate strongly with condition difficulty—but that they answer a different question. Ranking conditions by difficulty is not the same as attributing failure behavior to a specific controllable intervention. The information that FailureOps adds is not a better correlation coefficient; it is the rescued and induced transition semantics that connect interventions to logical outcomes on the same physical shots.

5.4 Runtime Replay Closure

The fourth intervention family applies a measured-runtime deadline to decoder corrections. We use decoder service-time measurements collected on the same real detector records (mean service time 4.65 μs, p95 5.64 μs) and apply a paired deadline intervention: if the measured service time for a shot exceeds a deadline budget, the correction is treated as late and dropped to a default no-flip correction. We evaluate five deadline budgets in $\{4, 5, 6, 7, 8\}$ μs on 10,000 paired shots.

Table 4. Runtime deadline intervention results. Mean measured service time is 4.65 μs, p95 is 5.64 μs.

Deadline	Miss rate	Baseline LFR	Intervened LFR	ΔLFR	Holm
4 μs	0.936	0.040	0.347	+0.307	
5 μs	0.194	0.040	0.103	+0.064	
6 μs	0.013	0.040	0.044	+0.004	
7 μs	0.000	0.040	0.040	0.000	
8 μs	0.000	0.040	0.040	0.000	

Table 4 reports the results. The effect is sharply thresholded. A 4 μs deadline causes a 93.6% miss rate—93.6% of corrections arrive too late—and induces a paired delta LFR of +0.307, with 185 rescued and 3,257 induced failures. This is the only deadline condition that is individually Holm-significant. At 5 μs, the miss rate drops to 19.4% and the paired delta LFR to +0.064 (32 rescued, 668 induced). At 6 μs, only 1.3% of corrections miss the deadline, and the delta LFR is negligible at +0.004. At 7 μs and above, the miss rate reaches zero and no logical-failure effect remains.

The threshold behavior—the sharp transition from a large effect at 4 μs to near-zero effect at 6 μs—is a direct consequence of the decoder service-time distribution. The mean service time (4.65 μs) lies between the 4 μs and 5 μs budgets, and the p95 (5.64 μs) sits between the 5 μs and 6 μs budgets. A system operator who must set a decoder deadline based on measured service time faces a genuine engineering trade-off: a 4 μs budget guarantees fast cycle times but induces substantial logical failures; a 6 μs budget eliminates the induced failures but requires longer cycle times; the 5 μs budget represents a gray zone.

We call this measured-runtime replay closure, not live control-stack timing attribution. The service-time measurements are collected on real detector records, but they are not obtained from an actual quantum control stack with hardware feedback latency, production queueing, or real-time scheduling. The replay demonstrates that decoder service time can be turned into a paired intervention whose effect on logical outcomes can be quantified on the same shot records. Closing this loop with live-hardware timing traces remains future work.

5.5 External and Expanded Evidence

External replication on qec3v5. The Google qec3v5 2022 surface-code scaling dataset provides an independent test of the decoder-pathway sensitivity result. The qec3v5 dataset uses a different code family (rotated surface code, distance 5), a different decoder baseline (PyMatching), and a different experimental setup (syndrome sampling at 13 cycle depths from

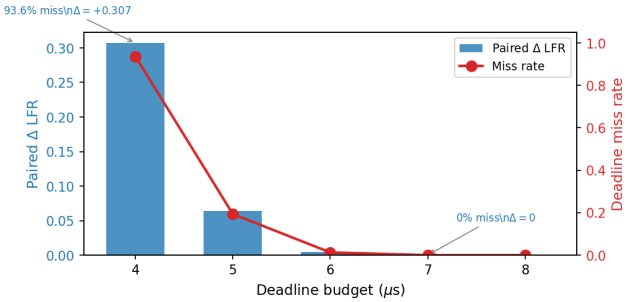

Figure 3. Paired delta LFR (bars) and deadline miss rate (line) as a function of decoder deadline budget. A sharp threshold appears between 4 μs and 6 μs, corresponding to the region around the measured mean service time of 4.65 μs.

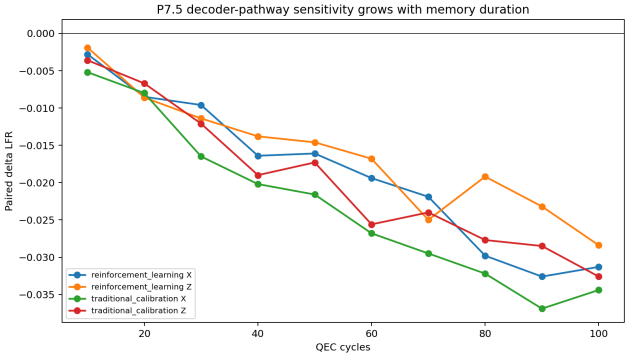

Figure 4. Paired delta LFR by cycle depth on the qec3v5 surface-code dataset. The effect strengthens monotonically with memory duration and is consistent across X and Z bases.

$r = 1$ to $r = 25$). We apply the same paired counterfactual design, switching from PyMatching to a correlated-matching decoder on 10,000 paired shots per condition.

Of the 26 conditions (13 X-basis, 13 Z-basis), 23 are individually significant after scope-wise Holm correction. The mean paired delta LFR is −0.0279 for X-basis conditions and −0.0268 for Z-basis conditions. The three non-significant conditions are the shortest-cycle conditions ($r = 1$ for X and Z, where the baseline LFR is below 0.01), which is consistent with the observation that intervention sensitivity grows with memory duration. Table 5 summarizes the cycle-level breakdown: the effect strengthens monotonically from $r = 1$ ($\Delta = -0.0001$, not significant) to $r = 11$ ($\Delta = -0.036$) and remains negative through all higher cycle settings. This external replication, on a different dataset with a different decoder baseline, supports the claim that decoder-pathway sensitivity is not an artifact of a particular code, decoder, or experiment design.

Table 5. qec3v5 external replication: paired delta LFR by cycle depth.

Cycles	Conditions	Mean ΔLFR	Holm sig.
1	2	−0.0001	0/2
3–9	8	−0.0219	8/8
11–25	16	−0.0333	15/16
All	26	−0.0274	23/26

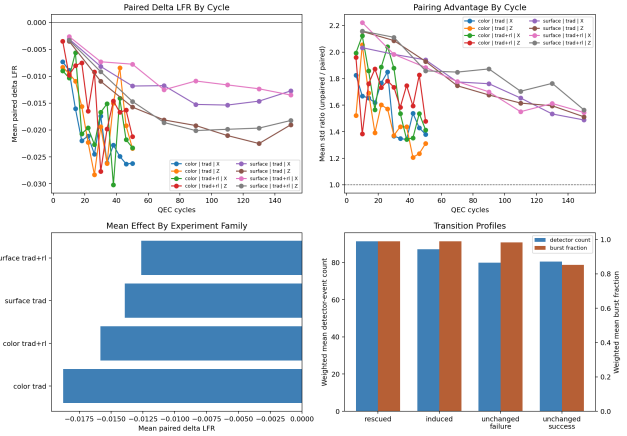

Figure 5. Expanded Google RL QEC v2 evidence: per-condition paired delta LFR across 496 conditions (top left), paired-vs-unpaired variance ratio (top right), subgroup aggregation (bottom left), and rescued/induced transition profiles (bottom right).

Expanded same-family stability on Google RL QEC v2. The Google RL QEC v2 corpus expands the original 40-condition matrix to 496 conditions with 4.96 million paired shots, adding color-code memory experiments and a second control mode (traditional_calibration_and_rl_fine_tuning). The overall mean paired delta LFR is −0.014, and 461 of 496 conditions (92.9%) are net-rescuing. After scope-wise Holm correction, 273 conditions are individually significant.

Table 6 breaks down the effect by subgroup. The mean paired delta is negative in every observed subgroup, with no reversal. Color-code memory conditions show a slightly stronger mean effect (−0.017) than surface-code memory (−0.013). Z-basis conditions (−0.016) show a moderately larger effect than X-basis (−0.011). The two control modes—traditional_calibration and traditional_calibration_and_rl_fine_tuning—produce nearly identical mean deltas (−0.014 and −0.013 respectively). The subgroup stability reinforces the main result: the paired sensitivity signal is not confined to a particular code, basis, or calibration mode but appears broadly across the expanded public real-record corpus.

This v2 expansion broadens the Google public corpus substantially, but it remains same-family evidence: both the

Table 6. Google RL QEC v2 subgroup breakdown. All subgroups show negative mean paired delta LFR.

Subgroup	Conditions	Mean ΔLFR
Surface code memory	448	−0.013
Color code memory	48	−0.017
Traditional calibration	248	−0.014
RL fine tuning	248	−0.013
X basis	248	−0.011
Z basis	248	−0.016
All v2 conditions	496	−0.014

original and v2 datasets come from the Google RL QEC release. We treat this as expanded stability evidence rather than independent cross-platform replication.

Corpus-level decoder-prior study. Scaling further, we evaluate whether the decoder-prior sensitivity observed on the SI1000-to-frequency-calibrated switch generalizes across prior variants within a fixed decoder family. The decoder-prior corpus study intervenes on three prior families—dem_correlations, dem_rl_isolated_correlated_matching, and dem_rl_shared_correlated_matching—each compared against a common dem_simple baseline, covering 1,908 real-data conditions per family. The total corpus spans 5,724 prior variants with 19.08 million paired shots.

Of the 5,724 variants, 4,879 (85.2%) yield bootstrap confidence intervals that exclude zero. The mean paired delta LFR is negative in all three prior families: −0.0133 (dem_correlations), −0.0212 (dem_rl_isolated), and −0.0180 (dem_rl_shared). The negative-delta fraction reaches 0.983, 0.997, and 0.960 respectively. The strongest single prior variant achieves a paired delta of −0.0494.

This result serves two purposes in the FailureOps evaluation. First, it demonstrates scale: the paired counterfactual pipeline can be applied to thousands of intervention variants without modification, producing comparable sensitivity metrics across an intervention design space. Second, it shows that decoder-prior sensitivity is not a single binary switch but a spectrum: different prior designs produce different effect magnitudes while preserving the negative sign. For a systems audience, this is analogous to a parameter-sensitivity study over a scheduler or allocator design space—the intervention becomes a tunable family rather than a binary treatment.

6 Discussion

6.1 Scope and Boundaries

FailureOps is evaluated on public real QEC detector records, which is both a strength and a boundary. The use of real records means the evaluation operates on data that a real QEC system produces, avoiding the external-validity concerns of pure simulation. However, all current evidence

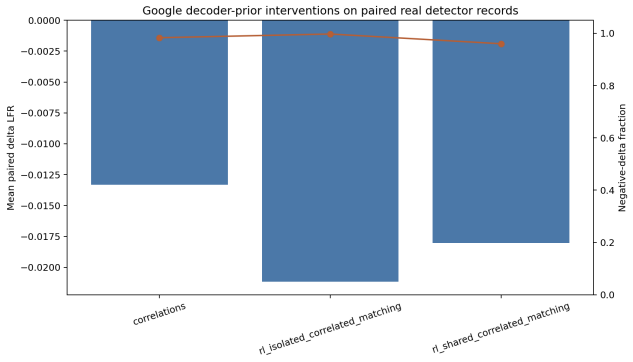

Figure 6. Distribution of paired delta LFR across 5,724 decoder-prior variants. The histogram peak lies in the negative region, confirming that most alternative prior configurations rescue more failures than the baseline SI1000 prior.

comes from Google's public QEC releases. The external replication on qec3v5 and the v2 same-family expansion broaden the evidence base, but they remain within the Google QEC ecosystem. Independent replication on non-Google detector records—from IBM, Quantinuum, or other platforms—is not yet available and remains future work.

The runtime-replay intervention is a measured-service-time closure, not a live-hardware attribution. The decoder service times are measured on the same detector records used for the logical outcome evaluation, which means the deadline intervention accurately reflects how long the decoder actually took. However, in a live system, decoder service time is only one component of the control-stack latency budget; hardware feedback, queueing delay, and real-time scheduling add additional sources of timing variation that our replay does not capture. We present the runtime intervention as evidence that decoder timing can be turned into a paired intervention, not as a claim that the specific deadline thresholds observed here would hold under live hardware conditions.

6.2 Generality of the Paired Design

The paired counterfactual design is not specific to decoder-pathway or prior interventions. Any component of the QEC pipeline that can be varied while holding the detector records fixed is a candidate intervention. Potential extensions include: varying the syndrome-extraction schedule (changing which stabilizers are measured in which rounds), applying different post-selection or erasure-conversion policies, testing the effect of decoding latency on logical outcomes under different code distances, and evaluating the sensitivity of failure behavior to calibration drift over time. The FailureOps methodology provides the measurement framework; the space of intervenable factors is limited only by what can be varied without changing the underlying physical execution records.

6.3 From Replay to Live Attribution

The natural next step for FailureOps is closing the gap from replay-based evidence to live-system attribution. This requires access to a QEC testbed that can log detector records, vary runtime parameters between shots, and measure decoder service time under real control-stack conditions. As experimental platforms from IBM, Google, and Quantinuum advance toward real-time decoding with logged detector streams, we expect that the paired counterfactual design will remain applicable: the same detector records, processed under different runtime configurations, with rescued and induced transitions reported at the shot level. The methodology does not change; the input data improves.

6.4 Statistical Considerations

The scope-wise Holm correction is a pragmatic choice that balances claim separation against multiplicity control. An alternative would be a global correction across all 6,291 tests, which would penalize the main decoder-pathway claim for the presence of thousands of prior-variant tests that address a different research question. We believe scope-wise correction is appropriate when the analysis scopes correspond to genuinely distinct families of claims, as they do here. However, readers who prefer a more conservative multiplicity adjustment can apply a global correction without affecting the sign-consistency evidence: the fraction of net-rescuing conditions and the paired-vs-unpaired variance ratio are descriptive statistics that do not depend on hypothesis-testing thresholds.

7 Related Work

7.1 QEC Evaluation and Benchmarking

The QEC community has developed a rich set of evaluation methodologies, from threshold estimation and logical error rate measurement to detailed error-source decomposition. Google's RL QEC experiments [**?**] and the qec3v5 surface-code scaling study [**?**] established public benchmarks for decoding performance on real detector records. Stim [**?**] and PyMatching [**?**] provide high-performance simulation and decoding backends. These tools and datasets are essential infrastructure, and FailureOps depends on them as inputs. The distinction is one of purpose: existing QEC evaluation measures how well a code and decoder perform, while FailureOps measures which controllable factors change that performance on the same execution records.

7.2 Error Classification and Root-Cause Analysis

A substantial body of work classifies QEC errors by physical origin: data-qubit errors, measurement errors, leakage, idling errors, and crosstalk. These classifications are diagnostically valuable and can inform code design and physical error mitigation. However, error classification answers a different question than FailureOps. Classifying a failure as measurement-error-related does not tell you whether a different decoder prior would have corrected it. Classifying a shot as idle-error-dominant does not tell you whether a shorter inter-cycle gap would have changed the outcome. FailureOps does not compete with error classification; it adds an orthogonal dimension—intervention sensitivity—that is not derivable from error-type labels alone.

7.3 Counterfactual Methods in Systems

Counterfactual reasoning has a long history in systems research. A/B testing, canary deployments, and shadow traffic evaluation are standard practice in distributed systems and networking. Causal inference frameworks, including difference-in-differences, instrumental variables, and propensity-score matching, have been applied to performance debugging [**?**], configuration tuning, and capacity planning. What distinguishes FailureOps from these methods is the domain constraint: in QEC, the detector record is the only observable output of a physical execution, and the decoder is a deterministic function of that record. The paired counterfactual is exact because the same detector record can be replayed through different configurations without resampling. This is a stronger guarantee than is typically available in classical systems counterfactual work, where re-running the same request through a different configuration is not always possible.

7.4 Reproducibility and Artifact Evaluation

EuroSys has a strong tradition of artifact evaluation and reproducible research. FailureOps is designed with reproducibility as a first-order requirement: the pipeline runs on public data, produces paper-facing tables from a single command, and preserves intermediate artifacts as human-readable CSV files. The claim-audit table explicitly maps each paper claim to its source artifact. We view this transparency as essential for a methodology paper whose primary contribution is a measurement framework rather than a system implementation.

8 Conclusion

This paper presented FailureOps, a paired counterfactual attribution methodology for logical failure behavior in quantum error correction. FailureOps evaluates the same shot-level detector records under a baseline and an intervened configuration, measures rescued and induced logical-failure transitions, and reports which controllable interventions change failure behavior on which classes of executions. The attribution is tied to the intervention, not to a static error label.

We instantiated FailureOps on public real QEC detector records from four datasets and three intervention families. The main decoder-pathway intervention rescues logical failures in all 40 observed real-data conditions, with 39 of 40

individually significant after scope-wise Holm correction. Pairing reduces estimator standard deviation by a factor of 1.75–1.78 relative to unpaired resampling. A corpus-level decoder-prior study across 5,724 prior variants confirms that the effect is not a single-condition anecdote but a replicable intervention-family phenomenon. The result externally replicates on the qec3v5 surface-code dataset (23 of 26 Holm-significant) and remains broadly net-rescuing on an expanded 496-condition corpus. Measured decoder service time can be closed into a paired deadline intervention with a sharp threshold near 5 μs.

FailureOps does not propose a new decoder, code, or threshold-estimation method. It provides a measurement framework for a question that the systems community will need as fault-tolerant quantum computation becomes an engineering discipline: given a protected execution, what controllable factor, if changed, would have altered its logical failure behavior?